Internship Report DeVi Comfort. 2026; 1(1): 1–10

DEVELOPING A REAL-TIME 3D TELEMETRY VISUALIZATION FOR STAIRLIFT FAULT DIAGNOSIS

Nout Mulder
DeVi Comfort • Engineering • Opmeer

Abstract. This research addresses the challenge of transforming raw stairlift telemetry data into an intuitive 3D visualization for fault diagnosis. The core technical problem is the real-time mapping of noisy encoder, spindle, and swivel sensor readings onto a curved 3D rail geometry, requiring state estimation, spline interpolation, and quaternion-based rotation. A "virtual ghost" prototype was developed iteratively and validated through calibration runs on physical hardware. The results show that a pipeline combining Kalman filtering, piecewise encoder mapping, and Catmull-Rom spline interpolation produces a stable, real-time 3D representation that enables faster interpretation of stairlift position and fault conditions compared to text-based error codes.

1 Introduction

1.1 Context

DeVi Comfort designs and manufactures stairlift systems and develops the accompanying control software in-house. A stairlift consists of a chair unit that travels along a custom-bent rail mounted to a staircase. The chair is driven by a traction motor, while a spindle motor adjusts seat tilt and a swivel motor rotates the seat at landing positions. During operation, embedded controllers continuously produce telemetry: traction encoder pulses, spindle encoder values, swivel angles, and system status bits including error codes.

For service, maintenance, and further development, engineers need to quickly understand the current state of a stairlift: where the chair is positioned on the rail, whether it is moving correctly, and what deviations or faults have occurred.

1.2 Problem

Fault information from the stairlift controller is currently presented as textual error codes and numeric status values. While technically correct, these representations require significant domain knowledge to interpret spatially. An error code indicating a position deviation, for example, does not immediately convey where on the rail the deviation occurred, in which direction, or how large the discrepancy is.

The underlying technical challenge is twofold. First, the raw telemetry data (encoder pulse counts, spindle values, swivel angles) must be mapped onto a three-dimensional rail geometry that contains straight sections, vertical bends, and horizontal bends, each with different encoder pulse costs. Second, the sensor readings arrive at irregular intervals over a Bluetooth serial link with noise, latency, and occasional packet loss, requiring real-time state estimation to produce a smooth and accurate visualization.

1.3 Relevance

A reliable 3D visualization of stairlift position and movement has direct practical value. Service engineers can visually identify whether the chair has lost its expected position, is stuck in a bend, or exhibits abnormal tilt. This reduces diagnosis time and minimizes unnecessary on-site visits. For the development team, a live telemetry view enables rapid verification of firmware behavior during testing and commissioning of new rail configurations.

1.4 Research Goal

The goal of this research is to determine which data processing and mathematical models are needed to transform live stairlift telemetry into a reliable and intuitive real-time 3D visualization for fault analysis.

The central research question is:

What data processing pipeline and mathematical models are required to transform live stairlift telemetry into a real-time 3D visualization that enables faster and more intuitive fault diagnosis compared to text-based error codes?

1.5 Approach

The research follows an iterative prototyping approach [12]. Each iteration consists of analysis, design, implementation, and validation against practical scenarios on physical hardware. The primary methods employed are formal analysis (mathematical derivation of filtering and interpolation models), architectural design (data pipeline and protocol

structure), and experiment (calibration run validation with measurable accuracy criteria).

1.6 Reading Guide

Section 2 presents the problem statement, structured as a problem analysis, prior work survey, and the resulting research questions. Section 3 describes the research methods per subquestion. Section 4 presents the findings. Section 5 concludes with answers to the research questions and recommendations.

2 Problem Statement

2.1 Problem Analysis

The visualization of mechatronic system state from sensor telemetry is a well-studied challenge in robotics, industrial automation, and vehicle diagnostics [8]. The core difficulty lies in bridging the gap between raw sensor data (discrete, noisy, asynchronous) and a continuous spatial representation that a human operator can interpret at a glance [2].

In the case of a stairlift, the rail geometry introduces additional complexity. Unlike a straight linear actuator where encoder pulses map linearly to position, a stairlift rail contains vertical bends (pitch changes), horizontal bends (yaw changes), and straight sections, each consuming a different number of encoder pulses per unit of travel. A single encoder reading is therefore insufficient to determine the 3D position without knowledge of the full rail topology and the per-segment pulse cost.

Furthermore, telemetry arrives over a Bluetooth Classic serial link [1] at approximately 20 Hz with variable latency. Between measurement updates, the visualization must extrapolate the chair position to maintain smooth rendering at 60 fps. Naive approaches such as directly plotting the most recent encoder value result in visible jitter and discontinuities, particularly during transitions between rail segments with different curvature.

Three interrelated technical subproblems emerge:

1. **Encoder-to-position mapping:** Converting a scalar encoder count into a normalized position along a 3D curve, accounting for segment-dependent pulse costs.
2. **State estimation:** Filtering noisy, asynchronous sensor readings to produce a smooth, low-latency estimate of position, velocity, and orientation.
3. **3D orientation reconstruction:** Determining the full spatial orientation (yaw, pitch, seat tilt, swivel angle) of the chair from multiple independent sensor streams.

2.2 Prior Work

2.2.1 State Estimation

For real-time state estimation from noisy sensor data, the Kalman filter is the standard approach in control engineering and robotics [9, 14]. The filter maintains a probabilistic estimate of system state (position and velocity) and updates it with each new measurement, optimally balancing prediction uncertainty against measurement noise [3]. Alternatives include simple moving average filters and complementary filters. Moving averages introduce latency proportional to the window size and cannot predict between measurements. Complementary filters combine two sensor sources (e.g., accelerometer and gyroscope) but assume complementary frequency characteristics, which does not apply to a single-encoder system. The Kalman filter is therefore the most suitable choice for this application: it provides both filtering and inter-measurement prediction in a single framework.

2.2.2 Spline Interpolation for Smooth Trajectories

To produce smooth position and orientation transitions along a discrete set of rail control points, interpolation is required. Linear interpolation produces sharp direction changes at segment boundaries. Cubic B-splines offer smoothness but do not pass through the control points, complicating correspondence with physical rail geometry. Catmull-Rom splines pass through each control point and provide C^1 continuity (smooth tangent transitions), making them a natural choice for trajectory following along a known set of waypoints [4].

2.2.3 Rotation Representation

Representing 3D orientation requires choosing between Euler angles, rotation matrices, and quaternions. Euler angles suffer from gimbal lock at certain configurations and require careful ordering of rotation axes [13]. Rotation matrices are gimbal-lock-free but costly to interpolate. Quaternions provide compact, gimbal-lock-free representation with efficient interpolation (SLERP) and are the standard in real-time 3D applications [10].

2.2.4 Existing Work at DeVi Comfort

Prior to this research, fault information within DeVi Comfort was presented exclusively as text-based error codes and numeric register values. No spatial visualization of stairlift telemetry existed. The IPC (Inter-Process Communication) infrastructure connecting the configurator application to the lift controller was limited to rail upload and download; live telemetry streaming had not been implemented.

2.3 Research Questions

Based on the problem analysis and prior work, the following subquestions are formulated. Together, they address the central research question.

1. Which fault types and status indicators from the stairlift controller are most relevant for spatial visualization, and how frequently do they occur in practice?
2. What is the required data pipeline architecture to transport live telemetry from the lift controller to a 3D rendering environment with sufficient throughput and reliability?
3. How can encoder pulse counts be mapped to a normalized position along a 3D rail geometry that contains segments with different curvature and pulse costs?
4. Which mathematical models for state estimation, interpolation, and rotation produce a stable and accurate real-time 3D chair representation from noisy telemetry?
5. Which rendering techniques make position deviations and fault conditions most interpretable in a 3D environment?
6. How can the prototype be validated against practical scenarios, and what measurable criteria determine whether the visualization is sufficiently accurate and responsive?

3 Methods

Each subquestion is addressed with one or more research methods. This section describes, per subquestion, which method is used, why it is appropriate, how it is concretely applied, and what criteria determine completion.

3.1 Fault Relevance (Observation Study)

Method: Observation study of the existing telemetry protocol.
Justification: The stairlift controller firmware defines over 168 message types. To determine which are most relevant for visualization, the existing protocol documentation and firmware behavior must be systematically catalogued.
Approach: The binary protocol specification is reverse-engineered from the firmware message definitions. Each telemetry message type is classified by content (position, speed, angle, status, error) and polling frequency. Relevance is assessed based on spatial interpretability and diagnostic value.
Instrument: A classification table listing each message type, its data fields, update rate, and relevance score for spatial visualization.

3.2 Data Pipeline Architecture (Design)

Method: Architectural design.
Justification: The telemetry pipeline spans multiple layers (Bluetooth transport, binary protocol parsing, coroutine-based polling, signal dispatch) and must satisfy real-time constraints. A structured architectural design ensures that each layer is independently testable and that throughput requirements are met.
Approach: The architecture is designed as a layered pipeline: transport abstraction, protocol parser, telemetry poller, state estimator, and 3D renderer. Each layer is specified with its inputs, outputs, timing constraints, and failure modes.
Instrument: A layered architecture diagram with throughput and latency specifications per layer.

3.3 Encoder-to-Position Mapping (Formal Analysis)

Method: Formal mathematical analysis.
Justification: The encoder-to-position mapping is a deterministic geometric problem: given a rail composed of segments with known curvature and known pulse costs per segment type, the mapping can be derived analytically. A formal derivation ensures correctness and makes the mapping reproducible.
Approach: The rail is modeled as a sequence of segments (straight, vertical bend, horizontal bend). For each segment type, the encoder pulse cost is derived from firmware constants. Cumulative 3D Euclidean distances are computed at segment boundaries, creating a piecewise linear mapping from encoder value to normalized rail position. The derivation includes the arc-tracing geometry used to construct 3D control points from the firmware's segment description.
Instrument: The formal derivation is considered complete when: (a) the mapping is defined for all segment types, (b) a calibration procedure aligns encoder values with 3D distances, and (c) the mapping error is below 5 mm on a reference rail.

3.4 Mathematical Models (Formal Analysis)

Method: Formal mathematical analysis.
Justification: The state estimation, interpolation, and rotation models are grounded in established mathematical theory (Kalman filtering, Catmull-Rom splines, quaternion algebra). Formal derivation from first principles ensures that the models are correctly applied and that tuning parameters have a clear theoretical basis.
Approach: Three models are derived:

- A 2D constant-velocity Kalman filter for each telemetry axis (traction, spindle, swivel), including the process noise matrix derivation, measurement update equations, and inter-measurement dead-reckoning extrapolation.

- Catmull-Rom spline interpolation for tangent computation along the rail, paired with linear position interpolation on segments.
- Quaternion-based orientation reconstruction for pitch (from rail geometry), spindle compensation, and swivel rotation.

Instrument: Each model is presented as a complete mathematical derivation with named equations. Correctness is verified by comparing predicted positions against known reference positions on a test rail.

3.5 Rendering Techniques (Design + Experiment)

Method: Design combined with informal comparative evaluation.
Justification: The choice of rendering technique affects interpretability. Multiple approaches are prototyped and compared based on visual clarity and information density.
Approach: Several visualization techniques are implemented: a ghost chair model following the rail, semi-transparent ghost stairs for spatial context, a dither-clip effect for depth perception, and rail annotations for segment identification. Each technique is evaluated in context of a running stairlift scenario.
Instrument: A comparison matrix listing each technique, its purpose, visual characteristics, and suitability for fault identification.

3.6 Validation (Experiment)

Method: Experiment with measurable criteria.
Justification: The prototype must be validated on physical hardware to verify that the mathematical models and pipeline architecture produce correct results under real-world conditions (Bluetooth latency, mechanical tolerances, sensor noise).
Approach: A calibration run is executed on a physical stairlift installation. The lift is driven from bottom to top while the prototype records encoder positions, applies the mathematical models, and renders the 3D ghost in real time. At horizontal bends, compass-based angle corrections are measured and compared against expected values.
Instrument: The experiment is evaluated on: (a) positional accuracy (deviation between displayed and actual position at known reference points), (b) angle correction accuracy at horizontal bends, (c) visual smoothness (absence of jitter at 60 fps), and (d) end-to-end latency from sensor reading to display update.

4 Results

4.1 Fault Types and Telemetry Classification

Analysis of the stairlift firmware protocol reveals 168 message types, of which six are directly relevant for real-time spatial visualization. These are classified in Table 1.

The three high-frequency streams (traction, spindle, swivel) form the core input for the 3D visualization. They are polled in an alternating pattern to share the serial Bluetooth link: the fast loop alternates between traction and spindle every frame tick, while a parallel coroutine polls swivel at 50 ms intervals. The low-frequency streams (system status, footrest) update chair accessory states every 2 seconds.

Status bits relevant for fault visualization include: armrest position (left and right, from `data[14]` bits 0 and 1 of MSG 114), seatbelt state (bit 2), and footrest position (bit 2 of MSG 113, inverted logic). These are mapped directly to accessory animations on the 3D chair model.

4.2 Data Pipeline Architecture

The telemetry pipeline is structured as five layers, shown in Figure 1.

Transport layer. Abstracts Bluetooth Classic and TCP connections behind a common interface. The Bluetooth transport communicates with the HC-06 module on the lift controller. A 40-byte null flush clears the module's receive buffer before each session. Message counters increment from 1 to 200 and wrap around.

Protocol parser. A six-state finite state machine [7] processes the byte stream: `WAIT_START` → `MSG_NUMBER` → `COMMAND_ID` → `LENGTH` → `DATA` → `CHECKSUM`. The wire format is:

$$[\texttt{0x0A}][\text{MsgNr}][\text{RwId}][\text{CmdId}][\text{Len}][\text{Data}_{0..39}][\text{Chk}] \tag{1}$$

where the checksum is computed as:

$$\text{Chk} = (\texttt{0x0A} + \text{MsgNr} + \text{RwId} + \text{CmdId} + \text{Len} + \textstyle\sum \text{Data}_i) \bmod 256 \tag{2}$$

Invalid lengths (> 40) or checksum mismatches trigger a resync to `WAIT_START`.

Telemetry poller. Three concurrent coroutine loops share the serial link cooperatively:

- *Fast loop* (∼20 Hz): alternates traction (MSG 110) and spindle (MSG 111) requests each frame tick.
- *Swivel loop* (∼20 Hz): polls the swivel angle via the APP_TO_BUS multiplexed channel (MSG 168) at 50 ms intervals.
- *Slow loop* (0.5 Hz): polls system status (MSG 114) and footrest state (MSG 113) every 2 s.

Data parsing. The traction encoder is extracted as a 32-bit big-endian integer from bytes 1–4 of the

Table 1: Telemetry message types relevant for spatial visualization.

Message	Data	Rate	Use
MSG 110	Traction encoder (32-bit)	20 Hz	Rail position
MSG 111	Spindle encoder (16-bit)	20 Hz	Seat tilt angle
MSG 168	Swivel encoder (16-bit)	20 Hz	Seat rotation
MSG 114	System status bits	0.5 Hz	Arm/seatbelt state
MSG 113	Footrest status	0.5 Hz	Footrest position
MSG 44	Pilot state	On demand	Calibration run

```
+-----------------+    +-----------------+    +-----------------+    +-----------------+    +-----------------+
| Transport       | -> | Protocol Parser | -> | Telemetry Poller| -> | State Estimator | -> | 3D Renderer     |
| (Bluetooth/TCP) |    | (6-state FSM,   |    | (3 coroutines,  |    | (3x Kalman      |    | (Ghost Chair    |
|                 |    |  checksum)      |    |  20 Hz polling) |    |  Filter)        |    |  + Ghost Stairs)|
+-----------------+    +-----------------+    +-----------------+    +-----------------+    +-----------------+
     raw bytes          LiftMessage objs        typed signals        filtered pos/angle      60 fps display
```

Figure 1: Layered telemetry pipeline architecture from Bluetooth transport to 3D rendering.

MSG 110 response. The spindle encoder is a 16-bit value from bytes 1–2 of MSG 111. The swivel angle is computed from MSG 168 as:

$$\theta_{\text{swivel}} = \frac{(\texttt{data[3]} \ll 8 \mid \texttt{data[4]}) - \texttt{0x7FFF}}{18} \tag{3}$$

4.3 Encoder-to-Position Mapping

4.3.1 Rail Geometry Construction

The lift controller stores the rail as a sequence of typed blocks: straight sections, vertical bends (up/down), and horizontal bends (left/right), each measured via incremental encoders [3]. Each block specifies a type and an amount. To construct a 3D representation, these blocks are traced sequentially using arc geometry. Figure 2 shows the three segment types and their arc-tracing geometry.

PLACEHOLDER: rail_segment_types.png

Figure 2: The three rail segment types: straight (left), vertical bend (center), and horizontal bend (right), with their respective arc-tracing geometry and encoder pulse costs.

The direction vector at any point along the rail is determined by the current yaw (ψ) and pitch (ϕ):

$$\mathbf{d}(\psi, \phi) = \begin{pmatrix} -\sin\psi\cos\phi \\ \sin\phi \\ -\cos\psi\cos\phi \end{pmatrix} \tag{4}$$

For straight sections, the position advances by the section length along $\mathbf{d}$. For bends, each segment subtends a fixed angle $\Delta\alpha = 5°$ at a bend radius $r = 150$ mm. The arc length per segment is:

$$s = r \cdot \Delta\alpha_{\text{rad}} = 0.150 \cdot \frac{5\pi}{180} \approx 0.01309\,\text{m} \tag{5}$$

Position is updated using a trapezoidal midpoint approximation:

$$\mathbf{p}_{i+1} = \mathbf{p}_i + \frac{\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}}{2} \cdot \frac{s}{\left\| \frac{\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}}{2} \right\|} \tag{6}$$

where $\mathbf{d}_{\text{before}}$ and $\mathbf{d}_{\text{after}}$ are the direction vectors before and after the pitch or yaw increment.

4.3.2 Piecewise Encoder Mapping

Different rail segment types consume different numbers of encoder pulses per unit of travel. The firmware defines the following constants:

$$\begin{aligned} k_{\text{straight}} &= 3.361352\,\text{pulses/mm} \\ k_{\text{up}} &= 37.547\,\text{pulses/segment} \\ k_{\text{down}} &= 50.453\,\text{pulses/segment} \\ k_{\text{horiz}} &= 44.0\,\text{pulses/segment} \\ E_{\text{start}} &= 100\,\text{pulses (initial offset)} \end{aligned} \tag{7}$$

A piecewise linear mapping is constructed by computing, at each segment boundary, both the cumulative encoder value and the cumulative 3D Euclidean distance from the rail start. For an encoder reading E, the normalized position $t \in [0, 1]$ is found by:

1. Locating the segment $[E_i, E_{i+1}]$ containing E.
2. Linearly interpolating the 3D distance: $d = d_i + (d_{i+1} - d_i) \cdot \frac{E - E_i}{E_{i+1} - E_i}$
3. Normalizing: $t = d/d_{\text{total}}$

This approach accounts for the non-linear relationship between encoder pulses and physical distance that arises at bends.

4.3.3 Spindle Angle Lookup

The spindle encoder value is converted to a seat tilt angle using a 751-entry lookup table ported from the firmware's `Curve.h`. The table maps angles from 0.0° to 75.0° in 0.1° increments to corresponding encoder pulse counts. Given a raw encoder reading, a base offset of 10 000 is subtracted, and an 11-iteration binary search locates the corresponding angle:

$$\theta_{\text{spindle}} = \frac{\text{bisect}(\text{TABLE}, \max(0, E_{\text{raw}} - 10000))}{10} \tag{8}$$

4.4 Mathematical Models for State Estimation

4.4.1 Kalman Filter

A 2D constant-velocity Kalman filter [9, 14] is applied independently to each telemetry axis (traction position, spindle angle, swivel angle). The state vector is $\mathbf{x} = [p, v]^T$ (position and velocity).

Prediction step (time update with interval Δt):

$$\begin{aligned} p &\leftarrow p + v \cdot \Delta t \\ P_{00} &\leftarrow P_{00} + 2\Delta t \cdot P_{01} + \Delta t^2 \cdot P_{11} + \sigma_a^2 \cdot \tfrac{\Delta t^3}{3} \\ P_{01} &\leftarrow P_{01} + \Delta t \cdot P_{11} + \sigma_a^2 \cdot \tfrac{\Delta t^2}{2} \\ P_{11} &\leftarrow P_{11} + \sigma_a^2 \cdot \Delta t \end{aligned} \tag{9}$$

where σ_a is the process noise (acceleration standard deviation) and $\mathbf{P}$ is the covariance matrix. The process noise matrix corresponds to the continuous white-noise-jerk model:

$$\mathbf{Q} = \sigma_a^2 \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix} \tag{10}$$

Measurement update (correction with measurement z):

$$\begin{aligned} y &= z - p \\ S &= P_{00} + \sigma_m^2 \\ K_0 &= P_{00}/S \\ K_1 &= P_{01}/S \\ p &\leftarrow p + K_0 \cdot y \\ v &\leftarrow v + K_1 \cdot y \end{aligned} \tag{11}$$

where σ_m is the measurement noise standard deviation and $\mathbf{K}$ is the Kalman gain.

The tuning parameters per axis are listed in Table 2.

4.4.2 Dead-Reckoning Extrapolation

Between measurement updates, the display position is extrapolated from the Kalman filter state:

$$p_{\text{target}} = p_{\text{KF}} + v_{\text{KF}} \cdot t_{\text{elapsed}} \tag{12}$$

After 0.8 s without a measurement, velocity decays exponentially:

$$v \leftarrow v \cdot e^{-5.0 \cdot (t_{\text{elapsed}} - 0.8)} \tag{13}$$

The display position is then smoothed with a frame-rate-independent exponential filter [5]:

$$p_{\text{display}} = \text{lerp}(p_{\text{display}},\ p_{\text{target}},\ 1 - e^{-\alpha \cdot \Delta t}) \tag{14}$$

where α is the smoothing rate from Table 2.

4.4.3 Catmull-Rom Spline Tangent

To determine the chair's orientation along the rail, the tangent direction at the current position is computed using a Catmull-Rom spline [4] through the surrounding control points $\mathbf{p}_0 \ldots \mathbf{p}_3$:

$$\mathbf{T}(t) = \tfrac{1}{2}[(-\mathbf{p}_0 + \mathbf{p}_2) + 2(2\mathbf{p}_0 - 5\mathbf{p}_1 + 4\mathbf{p}_2 - \mathbf{p}_3)t + 3(-\mathbf{p}_0 + 3\mathbf{p}_1 - 3\mathbf{p}_2 + \mathbf{p}_3)t^2] \tag{15}$$

The position along segments uses linear interpolation rather than the full Catmull-Rom curve, ensuring the chair stays on the rail segments while rotating smoothly through transitions.

4.4.4 Quaternion Orientation

The chair yaw is computed from the tangent's horizontal projection [10]:

$$\psi = \text{atan2}(T_x, T_z) \tag{16}$$

with phase unwrapping to avoid 2π discontinuities.

Rail chirality (which side the chair faces) is determined by the cumulative cross product of consecutive direction vectors projected onto the xz-plane:

$$C = \sum_i (d_x^i \cdot d_z^{i+1} - d_z^i \cdot d_x^{i+1}) \tag{17}$$

If $C \geq 0$, the rail is left-turning and the chair faces outward at +90°; otherwise at +270°.

The pitch tilt of the chair unit is applied via a world-to-local quaternion transform:

$$q_{\text{unit}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a}_{\text{pitch}},\ -\phi) \cdot q_{\text{parent}} \tag{18}$$

where $\phi = \text{atan2}(T_y, \|\mathbf{T}_{xz}\|)$ and $\mathbf{a}_{\text{pitch}} = \hat{\mathbf{y}} \times \hat{\mathbf{T}}_{\text{flat}}$.

The spindle motor compensates the rail pitch to keep the seat level, then adds its own tilt:

$$q_{\text{spindle}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a},\ -\phi + \theta_{\text{spindle}}) \cdot q_{\text{parent}} \tag{19}$$

A geometry blend factor of 15% mixes the expected pitch angle (from rail geometry) into the Kalman-filtered spindle angle:

$$\theta_{\text{target}} = 0.85 \cdot \theta_{\text{KF}} + 0.15 \cdot \phi_{\text{rail}} \tag{20}$$

Table 2: Kalman filter tuning parameters per telemetry axis.

Parameter	Traction	Spindle	Swivel
σ_a (process noise)	0.03	8.0	10.0
σ_m (measurement noise)	0.0002	0.2	0.5
Display smoothing rate	8.0	12.0	12.0
Max velocity	$1.2v_{\max}$	$30^\circ/\mathrm{s}$	$40^\circ/\mathrm{s}$

4.5 Rendering Techniques

Five visualization techniques were implemented in the Godot Engine [6] and evaluated for their contribution to fault interpretability:

1. **Ghost chair model:** A multi-pivot 3D model (unit, swivel, spindle, fold, frame) loaded from GLB parts, following the rail position in real time. The pivot hierarchy enables independent animation of each mechanical degree of freedom.
2. **Ghost stair renderer:** Semi-transparent stair treads generated from the rail height profile. Treads are placed every 180 mm of vertical rise, with even distribution between platform landings. This provides spatial context for the chair's position on the staircase.
3. **Dither-clip effect:** A shader-based clipping technique that gradually reveals or hides geometry based on distance from the chair, reducing visual clutter while maintaining context.
4. **Rail annotations:** Labels overlaid on the 3D rail identifying segment types, boundaries, and charging station positions.
5. **Liquid glass UI:** A frosted-glass shader design system for information panels, optimized for both desktop and tablet rendering.

Figure 3 shows the ghost chair visualization tracking the stairlift position in real time along a curved rail with ghost stairs.

The ghost chair model combined with ghost stairs proved most effective for spatial fault diagnosis: the chair's position on the staircase is immediately apparent, and deviations in tilt or position are visible as misalignment between the chair and the expected rail trajectory.

4.6 Validation Results

The prototype was validated through calibration runs on physical stairlift installations. During a calibration run, the lift is driven from bottom to top (and back) while the system polls telemetry at 250 ms intervals. At each horizontal bend, a compass-based angle correction is computed and compared against the expected value.

PLACEHOLDER: calibration_run.png

Figure 4: The calibration run in progress: the ghost chair tracks the physical lift position while compass-based angle corrections are applied at horizontal bends.

Calibration run protocol. The service polls six data points per tick: lock state, traction encoder, pilot state (keep-alive), spindle encoder, lift location, and (every 4th tick) error status. Speed is managed automatically: 50 (normal) for straight sections and 20 (reduced) for bends and short straights under 250 mm.

Angle correction. At horizontal bends, the compass tracker measures accumulated yaw via gyroscope integration:

$$\psi_{\mathrm{acc}} \mathrel{+}= \omega_z \cdot \frac{180}{\pi} \cdot \Delta t \tag{21}$$

The correction is computed as:

$$\delta = |\theta_{\mathrm{target}}/2| - |\psi_{\mathrm{measured}}| \tag{22}$$

where $\theta_{\mathrm{target}} = n_{\mathrm{segments}} \cdot 5^\circ$ is the expected total bend angle. This correction is transmitted to the firmware as a scaled 16-bit value ($\mathrm{round}(\delta \cdot 8.0)$).

Split-bend compensation. When two horizontal bends of the same direction are separated by a short straight section (< 250 mm), the first bend's accumulated yaw must be subtracted before measuring the second. The detector identifies this pattern and applies the correction automatically.

Validation outcomes.

- The ghost chair tracked the physical chair position with no perceptible lag during straight sections and smooth transitions through bends.
- The Kalman filter eliminated encoder jitter visible in raw telemetry, producing a stable display at 60 fps.
- The piecewise encoder mapping correctly accounted for different pulse costs at bends, preventing position jumps at segment boundaries.

Figure 3: The virtual ghost visualization: the 3D ghost chair follows the rail in real time based on live telemetry. Semi-transparent ghost stairs provide spatial context. Rail annotations identify segment boundaries along the rail.

- The spindle geometry blend (85% Kalman, 15% rail pitch) prevented oscillation in seat tilt visualization while maintaining responsiveness.
- The system performed reliably on both desktop (Linux) and tablet (Android 12+) platforms, with shadow and shader quality automatically adjusted for mobile hardware [11].

5 Conclusion

5.1 Answers to Research Questions

Fault types. Of the 168 firmware message types, six are directly relevant for spatial visualization. The three high-frequency streams (traction at 20 Hz, spindle at 20 Hz, swivel at 20 Hz) provide the positional and orientation data needed for the 3D ghost. Low-frequency status messages (0.5 Hz) supply accessory states. This classification enables a targeted polling strategy that maximizes information density within the bandwidth constraints of the Bluetooth serial link.

Pipeline architecture. A five-layer architecture (transport, protocol parser, telemetry poller, state estimator, 3D renderer) provides the required separation of concerns. The protocol parser's finite state machine handles byte-level reliability (checksumming, resync). The three concurrent polling coroutines achieve approximately 20 Hz update rates by cooperatively sharing the serial link. This layered design allows each component to be developed and tested independently.

Encoder mapping. A piecewise linear mapping, calibrated against cumulative 3D Euclidean distances at segment boundaries, correctly converts encoder pulse counts to normalized rail positions. The mapping accounts for the different pulse costs of straight sections (3.36 pulses/mm), upward bends (37.55 pulses/segment), downward bends (50.45 pulses/segment), and horizontal bends (44.0 pulses/segment). Without this piecewise approach, position errors accumulate at bends where the linear assumption breaks down.

Mathematical models. The combination of a 2D constant-velocity Kalman filter (with white-noise-jerk process model), Catmull-Rom spline tangent interpolation, and quaternion-based orientation reconstruction produces a stable real-time 3D representation. The Kalman filter provides both noise rejection and inter-measurement prediction. The Catmull-Rom tangent ensures smooth rotation transitions without requiring the position to deviate from the rail segments. Quaternion transforms avoid gimbal lock in the multi-axis rotation chain (yaw, pitch, spindle, swivel).

Rendering techniques. The ghost chair model with semi-transparent ghost stairs provides the most effective spatial context for fault diagnosis. The multi-pivot hierarchy enables independent visualization of each mechanical degree of freedom (traction position, unit tilt, seat tilt, swivel rotation, armrests, footrest). The dither-clip effect and rail annotations add supplementary context without occluding the primary visualization.

Validation. Calibration runs on physical hardware confirm that the pipeline produces accurate, smooth, real-time tracking. The Kalman filter eliminates visible jitter. The piecewise encoder mapping prevents position discontinuities at segment boundaries. The

compass-based angle correction at horizontal bends, including split-bend compensation, achieves accurate yaw alignment. The prototype runs at 60 fps on both desktop and tablet platforms.

5.2 Answer to Central Question

The data processing pipeline required to transform live stairlift telemetry into a real-time 3D visualization consists of five layers: Bluetooth transport, checksummed protocol parsing, multi-rate coroutine polling, Kalman-filter-based state estimation, and spline-interpolated 3D rendering. The mathematical foundation rests on three pillars: a constant-velocity Kalman filter for noise rejection and prediction, piecewise encoder mapping for non-linear rail geometries, and quaternion-based orientation from Catmull-Rom tangent vectors. This combination produces a visualization that is both spatially accurate and temporally smooth, enabling fault diagnosis that would require significant interpretation effort with text-based error codes alone.

5.3 Recommendations

- **Quantitative user study:** The current validation confirms technical correctness. A structured comparison measuring diagnosis time with and without the 3D visualization would quantify the practical benefit.
- **Historical playback:** Recording telemetry sessions for later playback would enable post-incident analysis without requiring a live connection.
- **Anomaly highlighting:** Automatic detection and visual emphasis of position deviations exceeding a configurable threshold would further reduce diagnosis time.
- **Extended sensor fusion:** Integrating accelerometer and gyroscope data from a tablet mounted on the chair could provide an independent position reference for continuous validation of the encoder-based estimate.

Bibliography

[1] Bluetooth SIG. Serial port profile. Adopted Specification v1.2, Bluetooth Special Interest Group, 2022. URL https://www.bluetooth.com/specifications/specs/serial-port-profile-1-2/.

[2] Jherson Bofill, Mideth Abisado, Jocelyn Villaverde, and Gabriel Avelino Sampedro. Exploring digital twin-based fault monitoring: Challenges and opportunities. *Sensors*, 23(16): 7087, 2023. doi: 10.3390/s23167087. URL https://www.mdpi.com/1424-8220/23/16/7087.

[3] William Bolton. *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*. Pearson Education, 7th edition, 2019. ISBN 978-1-292-25097-7. URL https://archive.org/search?query=bolton+mechatronics+7th.

[4] Edwin Catmull and Raphael Rom. A class of local interpolating splines. In R. E. Barnhill and R. F. Riesenfeld, editors, *Computer Aided Geometric Design*, pages 317–326. Academic Press, New York, 1974. doi: 10.1016/B978-0-12-079050-0.50020-5. URL https://doi.org/10.1016/B978-0-12-079050-0.50020-5.

[5] Rory Driscoll. Frame rate independent damping using lerp. Blog post, 2016. URL https://www.rorydriscoll.com/2016/03/07/frame-rate-independent-damping-using-lerp/.

[6] Godot Engine Contributors. Godot engine documentation, 2024. URL https://docs.godotengine.org/. MIT License, Version 4.x.

[7] Robert David Graham and Peter C. Johnson. Finite state machine parsing for internet protocols: Faster than you think. In *IEEE Symposium on Security and Privacy Workshops*, pages 185–190. IEEE, 2014. doi: 10.1109/SPW.2014.34. URL https://www.ieee-security.org/TC/SPW2014/papers/5103a185.PDF.

[8] Michael Grieves and John Vickers. Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In Franz-Josef Kahlen, Shannon Flumerfelt, and Anabela Alves, editors, *Transdisciplinary Perspectives on Complex Systems*, pages 85–113. Springer, Cham, 2017. doi: 10.1007/978-3-319-38756-7_4. URL https://research.fit.edu/media/site-specific/researchfaboraboratories/dmslab/documents/MG--AAIA-Draft2--2016-1-13.pdf.

[9] Rudolph Emil Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME – Journal of Basic Engineering*,

82(1):35–45, 1960. doi: 10.1115/1.3662552. URL https://www.cs.unc.edu/~welch/kalman/media/pdf/Kalman1960.pdf.

[10] Jack B. Kuipers. *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton University Press, 1999. ISBN 978-0-691-05872-6. URL https://press.princeton.edu/books/paperback/9780691102986/quaternions-and-rotation-sequences.

[11] Tomáš Marek and Ondřej Krejcar. Optimization of 3D rendering in mobile devices. In *Mobile Web and Intelligent Information Systems (MobiWIS 2015)*, volume 9228 of *Lecture Notes in Computer Science*, pages 37–48, Cham, 2015. Springer. doi: 10.1007/978-3-319-23144-0_4. URL https://doi.org/10.1007/978-3-319-23144-0_4.

[12] Ken Schwaber and Jeff Sutherland. The scrum guide: The definitive guide to scrum – the rules of the game, 2020. URL https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf.

[13] Ken Shoemake. Animating rotation with quaternion curves. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, pages 245–254. ACM, 1985. doi: 10.1145/325165.325242. URL https://www.ljll.fr/~frey/cours/references/quaternions.pdf.

[14] Greg Welch and Gary Bishop. An introduction to the Kalman filter. Technical Report TR 95-041, University of North Carolina at Chapel Hill, 1995. URL https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf. Updated 2006.